



TRABAJO PRÁCTICO INTEGRADOR. PROGRAMACIÓN 2

Food Store – Sistema de Gestión de Pedidos de Comida (Consola + JDBC)

Descripción breve

Aplicación interactiva de consola orientada a la gestión de pedidos de comida, desarrollada bajo los conceptos de la programación orientada a objetos en Java 21 y vinculada a una base de datos relacional MySQL a través de JDBC y

PINTO, Guillermo;SOTO, Matías;STORANI, Franco

Fecha entrega: 19/06/2026

Contenido

Introducción y Objetivos del Sistema	2
Marco Teórico	2
Decisiones Técnicas y Arquitectura en Capas	3
El Diseño de la Base de Datos Relacional	4
Manejo de Errores y Seguridad en las Compras	4
Dificultades y Soluciones	5
El Sistema en Funcionamiento y Conclusión	6
Bibliografía y Webgrafía	6

Introducción y Objetivos del Sistema

El proyecto Food Store nace de la necesidad de crear un sistema informático que permita administrar de forma ordenada un negocio de venta de comidas. Como estudiantes que estamos dando nuestros primeros pasos en programación, entendemos que un comercio de este tipo maneja mucha información al mismo tiempo, desde los productos que ofrece hasta los datos de los clientes y el control de cada pedido que se realiza en el día. El gran objetivo de este trabajo es aplicar las herramientas que aprendimos en la materia para construir una aplicación que funcione directo en la consola de la computadora, pero que sea capaz de guardar la información de manera real en una base de datos. De esta forma, el sistema ayuda a que el negocio no pierda el rastro de sus ventas, mantenga su menú actualizado y pueda registrar a sus usuarios sin errores humanos, automatizando cuentas que antes se hacían a mano.

Marco Teórico

Para construir el sistema Food Store nos apoyamos en un conjunto de tecnologías y conceptos de programación que nos permitieron pasar de un diseño en papel a una aplicación real y funcional. La base de todo nuestro proyecto es Java, un lenguaje de programación orientado a objetos que elegimos porque nos obliga a ser muy ordenados con las estructuras y los tipos de datos. Junto con Java, el núcleo del sistema es la Base de Datos Relacional, específicamente utilizando MySQL. A diferencia de nuestros primeros ejercicios de la materia donde los datos se borraban al cerrar la pantalla, la base de datos funciona como un archivador permanente en el disco de la computadora, donde la información se guarda en tablas conectadas entre sí mediante claves primarias y foráneas. Para lograr que nuestro código escrito en Java pudiera enviar instrucciones y recibir respuestas de la base de datos MySQL, implementamos JDBC, que es la tecnología de conectividad oficial de Java. Esta herramienta nos permitió abrir canales de comunicación estables y enviar comandos en lenguaje SQL de forma directa.

Por el lado de los conceptos de programación que nos pedía el diagrama UML, la estructura se sostiene sobre tres pilares de la programación orientada a objetos. El primero es la herencia, que aplicamos a través de la clase abstracta Base; esto sirvió para que clases como Usuario o Producto heredaran de forma automática características comunes como el identificador único o la marca de tiempo de creación, evitando tener

que reescribir ese código muchas veces. El segundo concepto es el uso de interfaces, reflejado en la interfaz *Calculable* que implementamos en la clase *Pedido*. Esto nos sirvió para definir un contrato obligatorio, asegurando que cualquier clase que deba realizar cuentas financieras tenga de manera obligatoria el método para calcular el total. Por último, aplicamos el polimorfismo, que es la capacidad de tratar a objetos de diferentes clases de una manera unificada gracias a que comparten una misma raíz. Esto se ve claramente cuando el sistema maneja listas de elementos que heredan de *Base*, permitiéndonos aplicar lógicas generales de guardado o borrado sin importar si estamos tratando con un cliente, una pizza o una categoría de comida.

Decisiones Técnicas y Arquitectura en Capas

La decisión técnica más importante que tomamos para abordar la solución fue separar el código en capas independientes utilizando el patrón arquitectónico DAO (Objeto de Acceso a Datos), lo cual nos ayudó a que las responsabilidades del sistema no se mezclaran e impidió que el código se volviera un lío imposible de entender. Organizamos el proyecto en seis paquetes principales dentro de la estructura de NetBeans, donde cada parte tiene una función muy específica para evitar conflictos.

La primera capa es la de vistas (*views*), que se encarga de mostrar los menús de texto en pantalla, hacer preguntas y recibir las respuestas que el operario escribe con el teclado. Cuando la vista recibe un dato, no lo procesa ella misma, sino que se lo pasa a la capa de servicios (*services*). En esta segunda capa se encuentra la lógica del negocio y las reglas que debemos cumplir obligatoriamente, como revisar si hay suficiente stock de un producto antes de aceptar el pedido, o comprobar si un correo electrónico ya fue registrado por otro cliente. Si la capa de servicios da el visto bueno y todo es correcto, la información viaja a la capa DAO. El paquete *dao* se comunica directamente con la base de datos MySQL, y allí escribimos las instrucciones en lenguaje SQL para guardar, modificar o traer la información utilizando la configuración centralizada del archivo *persistence.xml*.

Por último, el paquete *entities* contiene los moldes de datos puros que representan los objetos del negocio basados estrictamente en el UML (como *Usuario*, *Producto* o *Pedido*) que simplemente transportan los datos de una capa a otra; el paquete *enums*

almacena los tipos de datos fijos como los roles o las formas de pago; y el paquete exception agrupa todas nuestras alertas de error personalizadas. Esta división nos sirvió muchísimo porque si algo falla al guardar los datos, sabemos que debemos revisar el paquete DAO y no la pantalla que ve el usuario.

El Diseño de la Base de Datos Relacional

Para asegurar la persistencia de la información, diseñamos una base de datos relacional compuesta por cinco tablas principales conectadas entre sí, lo que evita que repitamos datos innecesariamente y mantiene todo organizado. La tabla de usuarios guarda los datos de los clientes y administradores, y se conecta con la tabla de pedidos, ya que un usuario puede realizar muchas compras a lo largo del tiempo. Los pedidos, a su vez, no guardan los productos directamente, sino que se dividen en una tabla llamada detalles de pedidos, la cual registra de forma individual cada renglón de la compra. Cada uno de estos renglones se conecta con un producto específico de la tabla de productos, detallando qué se compró y en qué cantidad. Por último, los productos se vinculan con una tabla de categorías, lo que nos permite clasificar la comida en secciones como bebidas, hamburguesas o postres.

Manejo de Errores y Seguridad en las Compras

Uno de los desafíos más grandes de este proyecto fue lograr que el sistema sea robusto y no se rompa ante el primer descuido del usuario. Para solucionarlo, creamos nuestras propias alertas mediante excepciones personalizadas. Si un operador intenta cargar un producto con un precio negativo o escribe un mail que ya existe en el sistema, el programa se da cuenta, frena la acción y lanza un mensaje de advertencia claro. En la capa de vistas pusimos bloques de captura de errores try-catch que atajan tanto nuestros propios avisos como las fallas de conexión a la base de datos. Gracias a esto, el programa nunca se cierra de golpe, sino que le explica al usuario qué hizo mal y lo devuelve al menú para que lo intente otra vez.

Además, implementamos una seguridad especial para el momento de confirmar una compra. Al registrar un pedido con múltiples productos, el sistema utiliza transacciones de la base de datos mediante comandos explícitos de commit y rollback. Esto significa que el pedido y todos sus renglones se envían juntos en un solo paquete. Si por alguna

razón el sistema se queda sin stock en la mitad de la carga o se corta la conexión, la base de datos ejecuta una orden de marcha atrás que borra lo que se había empezado a anotar. De esta manera, nos aseguramos de que no queden pedidos a medias ni registros incompletos, garantizando que una compra se guarda completa o directamente no se guarda nada.

Dificultades y Soluciones

El desarrollo de este integrador estuvo lleno de desafíos que nos pusieron a prueba constantemente, principalmente porque nuestros conocimientos previos en algunas tecnologías eran mínimos. El obstáculo más grande fue, sin dudas, la integración de la base de datos mediante JDBC. Al principio nos sentíamos completamente perdidos intentando entender cómo conectar Java con MySQL, cómo configurar el archivo de persistencia y cómo manejar los estados de las conexiones sin que el programa se tildara. Muchas veces las consultas SQL fallaban por un error de sintaxis menor o por no respetar el orden de los parámetros, lo que nos generaba bastante frustración al no saber si el problema estaba en nuestro código de Java o en el servidor de la base de datos.

Otra dificultad importante apareció al intentar entrelazar las clases del sistema. Al tener tantas entidades relacionadas unas con otras —como un Pedido que contiene Detalles, que a su vez conocen a un Producto, el cual pertenece a una Categoría—, la lógica se volvía muy enredada. En varias ocasiones nos pasó de perder el hilo de lo que estábamos haciendo, arrastrando objetos vacíos que hacían fallar el programa de golpe. Para superar estos momentos en los que nos encontrábamos trabados, la clave fue la colaboración grupal. Decidimos organizar sesiones de trabajo juntos para revisar con mucha paciencia el camino que recorre cada dato. Mientras uno compartía la pantalla y mostraba el código, los otros dos seguían la lógica atentos para corregir errores o proponer alternativas. Esta dinámica de ayudarnos mutuamente no solo nos permitió destrabar el código y entender por fin el uso de las capas, sino que nos enseñó el valor de trabajar en equipo cuando el desafío supera los conocimientos individuales de cada uno.

El Sistema en Funcionamiento y Conclusión

Cuando ponemos en marcha el programa, el menú principal nos recibe con opciones numeradas para navegar por las distintas áreas del negocio. Un administrador puede entrar a la sección de catálogo para dar de alta nuevas categorías y productos con sus respectivos precios y cantidades disponibles. También puede listar a los usuarios registrados del sistema. Para mantener la seguridad de la auditoría del negocio, cuando se decide borrar un producto o un cliente, no lo hacemos desaparecer físicamente de la base de datos. En su lugar, aplicamos una baja lógica, que consiste en cambiar un atributo interno que oculta ese elemento de las pantallas de navegación, pero lo mantiene guardado para los registros históricos del comercio.

Haber desarrollado este integrador nos demostró lo complejo y satisfactorio que es construir software real en equipo. Aunque al principio nos costó mucho coordinar las ramas de Git para no pisarnos el código y entender cómo conectar Java con una base de datos mediante JDBC, logramos entender el propósito de cada capa. Aprendimos que programar no es solo escribir líneas de código para que la pantalla se vea linda, sino diseñar una estructura lógica que sea ordenada para nosotros como desarrolladores, segura para los datos del negocio y fácil de usar para cualquier operador.

Bibliografía y Webgrafía

- **Videos de YouTube (Charly Cimino):** Cimino, C. [Charly Cimino]. (2020-2024). *Programación Orientada a Objetos en Java* [Lista de reproducción de Videos]. YouTube. <https://youtube.com/playlist?list=PLow7b-NX043aSC7ZNtEuVfY8xZoNzVqdJ&si=nvqf0Z9SfgkOpWul>
- **Material del Aula Virtual de la Universidad:** Universidad Tecnológica Nacional (UTN). (2026). *Material de lectura y apuntes de cátedra: Aula Virtual de Programación 2. Tecnicatura Universitaria en Programación.*
- **Documentación Oficial de Java (Oracle):** Oracle Corporation. (2026). *Java Platform, Standard Edition (Java SE) Documentation.* <https://docs.oracle.com/en/java/>

- **Documentación Oficial de MySQL:** Oracle Corporation. (2026). *MySQL Reference Manual (Connector/J and JDBC Reference)*. <https://dev.mysql.com/doc/>

Links al repositorio del proyecto.

- Repositorio Github: <https://github.com/MatiasSoto21/Food-Store-Prog2>